FRONT-END PRACTICAL ASSESSMENT COMPLETE PROJECT NOTES

React • Vite • JavaScript • Bootstrap • Fetch API • React Hook Form • React Router • Postman • Git

Offline-first reference guide | Windows + VS Code

0. Read this first: assessment strategy

- Follow the assessment brief and provided mockup/API documentation over these generic examples.
- Do not guess endpoint URLs, request fields, response shapes, or authentication. Copy them from the assessment instructions or confirm them in Postman.
- Build one small vertical slice first: display records from API → add record → edit record → delete record. Then improve the layout.
- Save often and commit working milestones with Git. Test after each major change.
- After internet disconnects, npm installs may fail unless dependencies are already in node_modules or npm cache. Prepare and test before the cutoff.

1. First 30 minutes: preparation checklist

1.1 Verify installed tools

```
node -v
npm -v
git --version
code --version
```

If a command is not recognized, resolve it while internet/help is still available. Confirm the provided project starts successfully before the cutoff.

1.2 Create project only if you need a fresh one

```
npm create vite@latest assessment-app -- --template react
cd assessment-app
npm install
npm install bootstrap react-hook-form react-router-dom
npm run dev
```

If a project is already provided, do not replace it. Open its folder, inspect package.json, run npm install while online if needed, then npm run dev.

1.3 Offline readiness

- Open the app in the browser and confirm it runs.
- Check package.json and ensure node_modules exists.
- Keep the assessment folder on the local drive; do not depend on cloud editors or online CDNs.
- Download any permitted documentation and API instructions during the allowed online period.
- Avoid deleting node_modules or running clean-install commands during the assessment unless you know the dependencies are cached.

2. Know the project structure

```
assessment-app/
■■ public/
■■ src/
■  ■■ assets/
■  ■■ components/
■  ■  ■■ Navbar.jsx
■  ■  ■■ Loading.jsx
■  ■  ■■ ConfirmDelete.jsx
■  ■■ pages/
■  ■  ■■ Dashboard.jsx
■  ■  ■■ ItemsPage.jsx
■  ■■ services/
■  ■  ■■ api.js
■  ■■ App.jsx
■  ■■ App.css
■  ■■ main.jsx
■  ■■ index.css
■■ package.json
■■ index.html
```

Folders such as components, pages, and services are not automatically created by Vite. Create them yourself in VS Code when your project needs them. Keep the structure simple if time is short.

3. Start the app and understand the entry files

```
npm run dev
# Stop the development server: Ctrl + C
```

main.jsx mounts the React app and imports global CSS. App.jsx usually contains the main page layout or routes. Component files return JSX.

3.1 main.jsx: import Bootstrap

```
import React from 'react';
import { createRoot } from 'react-dom/client';
import 'bootstrap/dist/css/bootstrap.min.css';
import 'bootstrap/dist/js/bootstrap.bundle.min.js';
import './index.css';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Bootstrap CSS gives you styling. The bundle JS is needed for interactive Bootstrap features such as dropdowns and modals. For simple layouts, CSS alone may be enough.

4. React essentials

4.1 Component and JSX

```
function Welcome({ name }) {
  return <h1 className="h3">Welcome, {name}!</h1>;
}

export default Welcome;
```

- Component names begin with a capital letter.
- Use className instead of class in JSX.
- Return one parent element, or wrap elements in <>...</>.
- Use {expression} to insert JavaScript values into JSX.
- For lists, use a stable key such as item.id.

4.2 State and events

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button className="btn btn-primary" onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}

export default Counter;
```

4.3 Controlled input

```
const [name, setName] = useState('');

<input
  className="form-control"
  value={name}
  onChange={(event) => setName(event.target.value)}
  placeholder="Enter name"
/>
```

4.4 Conditional rendering and lists

```
{loading && <p>Loading...</p>}
{error && <div className="alert alert-danger">{error}</div>}
{items.length === 0 ? <p>No records found.</p> : (
  items.map(item => <div key={item.id}>{item.name}</div>)
)}
```

5. Suggested App.jsx starter

```
import { useState } from 'react';

export default function App() {
  const [pageTitle] = useState('Assessment App');

  return (
    <div className="container py-4">
      <header className="mb-4">
        <h1 className="h2">{pageTitle}</h1>
        <p className="text-secondary">Manage records from the provided API.</p>
      </header>

      <div className="card shadow-sm">
        <div className="card-body">
          <h2 className="h5">Workspace</h2>
          <p className="mb-0">Replace this section with the required mockup.</p>
        </div>
      </div>
    </div>
  );
}
```

6. Bootstrap layout quick reference

```
<div className="container py-4">
  <div className="d-flex justify-content-between align-items-center mb-3">
    <h1 className="h3 mb-0">Items</h1>
    <button className="btn btn-primary">Add item</button>
  </div>

  <div className="row g-3">
    <div className="col-12 col-md-6 col-lg-4">
      <div className="card h-100">
        <div className="card-body">
          <h2 className="h5 card-title">Item title</h2>
          <p className="card-text">Item details</p>
          <button className="btn btn-outline-primary btn-sm">View</button>
        </div>
      </div>
    </div>
  </div>
</div>
```

- container / container-fluid: page width
- row + col-12 col-md-6 col-lg-4: responsive grid
- d-flex, justify-content-between, align-items-center: alignment
- mb-3, mt-4, p-3, gap-2: spacing
- btn btn-primary / btn-outline-secondary / btn-danger: buttons
- form-control, form-select, form-label: form controls
- table table-striped table-hover: data table
- alert alert-danger / alert-success: status messages
- badge text-bg-success: status badge

7. API integration using Fetch

Use the real base URL, endpoint paths, headers, field names, and response structure supplied during the assessment. The examples below use placeholders.

7.1 Centralize API URL and functions

```
// src/services/api.js
const API_BASE_URL = 'REPLACE_WITH_PROVIDED_API_BASE_URL';

async function request(path, options = {}) {
  const response = await fetch(`${API_BASE_URL}${path}`, {
    headers: {
      'Content-Type': 'application/json',
      ...(options.headers || {}),
    },
    ...options,
  });

  if (!response.ok) {
    const message = await response.text();
    throw new Error(message || `Request failed: ${response.status}`);
  }

  if (response.status === 204) return null;
  return response.json();
}

export const getItems = () => request('/REPLACE_WITH_ITEMS_ENDPOINT');

export const createItem = (data) => request('/REPLACE_WITH_ITEMS_ENDPOINT', {
  method: 'POST',
  body: JSON.stringify(data),
});

export const updateItem = (id, data) => request(`/REPLACE_WITH_ITEMS_ENDPOINT/${id}`, {
  method: 'PUT',
  body: JSON.stringify(data),
});

export const deleteItem = (id) => request(`/REPLACE_WITH_ITEMS_ENDPOINT/${id}`, {
  method: 'DELETE',
});
```

Important: if the API expects PATCH rather than PUT, change the method and send only the fields it expects. Some APIs wrap records in { data: [...] }; others return an array directly. Inspect the actual response and adapt the component.

7.2 HTTP methods

- GET: retrieve data; usually no request body.
- POST: create a record; send JSON in body.
- PUT: replace/update a resource, depending on API design.
- PATCH: update selected fields.
- DELETE: delete a resource; response may be empty (204).

7.3 Fetch directly: GET example

```
const response = await fetch(`${API_BASE_URL}/items`);
if (!response.ok) throw new Error(`HTTP ${response.status}`);
const data = await response.json();
console.log(data);
```

7.4 POST JSON example

```
const response = await fetch(`${API_BASE_URL}/items`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ name: 'Sample item', price: 100 }),
});
if (!response.ok) throw new Error(`HTTP ${response.status}`);
const created = await response.json();
```

8. Display API records with loading and error handling

```
// Example src/pages/ItemsPage.jsx
import { useEffect, useState } from 'react';
import { getItems, deleteItem } from '../services/api';

export default function ItemsPage() {
  const [items, setItems] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');

  async function loadItems() {
    try {
      setLoading(true);
      setError('');
      const result = await getItems();

      // Adapt this line to the real API response:
      setItems(Array.isArray(result) ? result : (result.data ?? []));
    } catch (err) {
      setError(err.message || 'Unable to load records.');
    } finally {
      setLoading(false);
    }
  }

  useEffect(() => {
    loadItems();
  }, []);

  async function handleDelete(id) {
    if (!window.confirm('Delete this record?')) return;
    try {
      await deleteItem(id);
      setItems(current => current.filter(item => item.id !== id));
    } catch (err) {
      setError(err.message || 'Delete failed.');
    }
  }

  if (loading) return <div className="container py-4">Loading...</div>;

  return (
    <div className="container py-4">
      <h1 className="h3 mb-3">Items</h1>
      {error && <div className="alert alert-danger">{error}</div>}
      <button className="btn btn-outline-secondary mb-3" onClick={loadItems}>
        Refresh
      </button>
      <div className="table-responsive">
        <table className="table table-striped align-middle">
          <thead><tr><th>Name</th><th>Actions</th></tr></thead>
          <tbody>
            {items.map(item => (
              <tr key={item.id}>
                <td>{item.name}</td>
                <td>
                  <button
                    className="btn btn-sm btn-danger"
                    onClick={() => handleDelete(item.id)}
                  >Delete</button>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
      {items.length === 0 && <p>No records found.</p>}
    </div>
  );
}
```

Change item.id and item.name to the actual property names returned by your API. If the endpoint uses a different ID key, update the delete call and React key too.

9. Forms with React Hook Form

```
import { useForm } from 'react-hook-form';

export default function ItemForm({ onSave }) {
  const {
    register,
    handleSubmit,
    reset,
    formState: { errors, isSubmitting },
  } = useForm({
    defaultValues: { name: '', price: '' },
  });

  async function submitForm(values) {
    await onSave({
      name: values.name.trim(),
      price: Number(values.price),
    });
    reset();
  }

  return (
    <form onSubmit={handleSubmit(submitForm)} noValidate>
      <div className="mb-3">
        <label className="form-label">Name</label>
        <input
          className={`form-control ${errors.name ? 'is-invalid' : ''}`}
          {...register('name', {
            required: 'Name is required.',
            minLength: { value: 2, message: 'Enter at least 2 characters.' },
          })}
        />
        {errors.name && <div className="invalid-feedback">{errors.name.message}</div>}
      </div>

      <div className="mb-3">
        <label className="form-label">Price</label>
        <input
          type="number"
          step="0.01"
          className={`form-control ${errors.price ? 'is-invalid' : ''}`}
          {...register('price', {
            required: 'Price is required.',
            valueAsNumber: true,
            min: { value: 0.01, message: 'Price must be greater than zero.' },
          })}
        />
        {errors.price && <div className="invalid-feedback">{errors.price.message}</div>}
      </div>

      <button className="btn btn-primary" disabled={isSubmitting}>
        {isSubmitting ? 'Saving...' : 'Save'}
      </button>
    </form>
  );
}
```

Parent component usage: pass an async onSave function that calls createItem(values), then reloads or updates the displayed records. Match the form fields to the assessment data model.

10. Search and filter

```
const [search, setSearch] = useState('');

const filteredItems = items.filter(item =>
  String(item.name ?? '').toLowerCase().includes(search.toLowerCase())
);

// In JSX:
<input
  className="form-control mb-3"
  placeholder="Search by name..."
  value={search}
  onChange={event => setSearch(event.target.value)}
/>

{filteredItems.map(item => <div key={item.id}>{item.name}</div>)}
```

For filtering by category or status, add another state value and include it in the filter condition. If API data is paginated, respect the pagination fields supplied by the API.

11. Routing with React Router

```
// App.jsx
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import ItemsPage from './pages/ItemsPage';
import Dashboard from './pages/Dashboard';

export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Dashboard />} />
        <Route path="/items" element={<ItemsPage />} />
        <Route path="*" element={<Navigate to="/" replace />} />
      </Routes>
    </BrowserRouter>
  );
}
```

```
// Navigation example
import { Link } from 'react-router-dom';

<Link className="nav-link" to="/">Dashboard</Link>
<Link className="nav-link" to="/items">Items</Link>
```

Only add routing if the mockup or requirements need multiple pages; a single-page CRUD app may not need it.

12. Postman workflow

- Create or open a collection for the provided API.
- Set the correct base URL and required headers/authentication.
- Test GET first; inspect status code, JSON shape, field names, and ID format.
- Test POST with the exact required JSON body.
- Test PUT/PATCH and DELETE using a real ID returned by the API.
- Record the working method, endpoint, headers, body, and expected response in your notes.
- A successful Postman request does not guarantee the browser can call it: browser requests can be blocked by CORS.

Common status codes

- 200 OK: successful request.
- 201 Created: resource created.
- 204 No Content: success with no response body.
- 400 Bad Request: invalid input/body.
- 401 Unauthorized: missing or invalid authentication.
- 403 Forbidden: not allowed.
- 404 Not Found: wrong URL or missing resource.
- 422 Unprocessable Entity: validation failure (common in some frameworks).
- 500 Internal Server Error: server-side failure.

13. Git from the beginning to submission

13.1 Initialize and first commit

```
git --version
git init
git status
git add .
git commit -m "Initial project setup"
git branch
```

If Git asks for identity, use the identity permitted by your assessment environment:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

13.2 Commit milestones

```
git status
git add src
git commit -m "Build main layout"
git add .
git commit -m "Integrate items API"
git log --oneline
```

13.3 Useful commands

```
git diff                # inspect uncommitted changes
git restore path/to/file # discard edits to one file (careful)
git branch              # list branches
git switch -c feature/form
git switch main
git merge feature/form
git log --oneline --graph --all
```

Do not run destructive commands such as git reset --hard unless you fully understand the consequence. Do not assume a remote repository is available; commit locally if the assessment is offline.

14. Debugging quick guide

- Blank page: inspect browser DevTools Console; check syntax, imports, component exports, and JSX.
- Vite says Missing script 'dev': open the correct project folder and inspect package.json scripts. A Vite project usually has "dev": "vite".
- Failed import: check exact filename, path, extension, capitalization, and default/named export.
- Cannot read properties of undefined: inspect actual API response and safely handle missing fields.
- Map error: ensure the value is an array before calling .map().
- API 404: verify base URL and endpoint path.
- API 400/422: inspect response body and match validation rules and property names.
- 401/403: check authentication and permissions.
- CORS error: browser blocked the cross-origin request; confirm the provided API is configured for the frontend origin. Postman does not enforce browser CORS.
- JSON parse error: response may be empty or HTML rather than JSON; inspect response status and text.
- Port in use: accept Vite's alternate port or stop the other server.
- npm install fails offline: do not keep retrying; use already installed dependencies or the prepared project.

15. Suggested build order under time pressure

– Read the brief; list pages, data fields, buttons, and required user actions.

– Check the provided API in Postman and write down each endpoint and request body.

– Start the app and build the outer layout/navigation based on the mockup.

– Implement GET and show loading, error, empty, and success states.

– Implement create form with validation.

– Implement edit/update if required.

– Implement delete with confirmation.

– Add search/filter/sort only if required or time permits.

– Compare the UI with the mockup at desktop and mobile widths.

– Test every button and API action.

– Check browser Console and Network tabs.

– Commit final working changes and verify the app starts with npm run dev.

16. Final testing checklist

- App starts with npm run dev.
- No red errors in browser Console.
- Layout matches the mockup and works at different widths.
- All required fields have labels and validation.
- Loading, error, empty, and success states are understandable.
- GET displays the correct records.
- Create/update/delete work against the provided API, where required.
- Search/filter controls work if included.
- Navigation works if there are multiple pages.
- No endpoint or field names are left as placeholders.
- Git status is understood and important work is committed.
- Project can run from the local folder without relying on internet resources.

17. Last-minute syntax sheet

```
// State
const [value, setValue] = useState('');

// Event
onClick={() => doSomething()}
onChange={event => setValue(event.target.value)}

// Effect
useEffect(() => { loadData(); }, []);

// Async
try { ... } catch (error) { setError(error.message); } finally { setLoading(false); }

// JSON request body
body: JSON.stringify(values)

// Render list
items.map(item => <div key={item.id}>{item.name}</div>)

// Conditional class
className={error ? 'form-control is-invalid' : 'form-control'}

// Git
git status
git add .
git commit -m "Describe the change"

// Run
npm run dev
```

18. Important reminder

This guide is a reusable framework, not a substitute for the actual assessment brief. The provided mockup and API documentation determine the real page names, field names, validation rules, routes, authentication, and response formats. Replace every placeholder before final testing.